

ProjectName – C/Java/Python

Rapport final

AIT-ISSAD MOHAMED SAID, BILAL AL FAYOUMI, AMARA KOUSSAY, GUILLEM CAUSS

9 avril 2026

Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Pellentesque iaculis molestie euismod. In hac habitasse platea dictumst. Duis eu gravida sem. Vivamus pulvinar, tellus auctor sollicitudin iaculis, quam massa blandit ligula, in mollis est tellus et velit. Nulla gravida sapien nibh, nec faucibus lorem aliquam sed. Curabitur rutrum elit purus, et congue neque venenatis vitae. Sed non magna nisi. Pellentesque euismod non ipsum a hendrerit. Pellentesque vehicula tincidunt lacinia. Sed rhoncus porttitor facilisis.

Etiam sed molestie odio. Nunc cursus sollicitudin est vitae posuere. Nullam pellentesque metus eu dui porttitor bibendum. Morbi egestas arcu ut urna dictum placerat non non nisl. Quisque non ante in eros cursus pulvinar. Sed augue libero, eleifend a urna eget, pellentesque condimentum odio. Sed sed dui non diam mattis interdum eget sed urna. Integer consectetur, erat id rutrum posuere, dui velit dictum mi, quis tincidunt magna diam sit amet nunc. Etiam dui augue, sollicitudin rhoncus nisl a, rhoncus tempor purus.

Suspendisse cursus erat et metus fermentum ullamcorper. Etiam eros arcu, dapibus quis efficitur ut, congue id nunc. Pellentesque aliquam venenatis convallis. Nulla nisi justo, aliquet quis lacus eget, aliquet maximus velit. Maecenas ex turpis, iaculis eget dignissim non, efficitur vel mi. Maecenas pharetra elit ornare sem suscipit ornare id id sapien. Vivamus sem neque, consectetur vitae aliquet non, finibus eu lectus. Fusce aliquam ipsum ut viverra vehicula. Donec ultrices varius nulla vel finibus. Donec convallis fringilla ante id porta.

1 Modélisation du jeu

1.1 Représentation de l'état du jeu

L'état d'une partie est centralisé dans `GameState`. La classe regroupe les positions des joueurs, les murs restants et posés, le graphe de connectivité et le joueur courant. Cette séparation entre état dynamique, contraintes de déplacement et progression du tour évite de disperser la logique métier.

Les positions utilisent un encodage linéaire :

$$node = row \times size + col.$$

Ce format simplifie l'accès aux structures indexées et les parcours. Les champs utilisés en exécution sont `board_size`, `current_player`, `player_positions`, `remaining_walls`, `vertical_walls`, `horizontal_walls` et `graph`. Cette représentation sert directement la validation des coups, l'historisation et la persistance.

1.2 Graphe du plateau et structures associées

Le plateau est modélisé par un graphe non orienté $G = (V, E)$: les sommets représentent les cases et les arêtes les déplacements élémentaires autorisés, conformément à [3, 6]. Le graphe est stocké en liste d'adjacence `dict[int, list[int]]`, adaptée à une grille creuse (degré maximal 4) avec un coût mémoire en $O(V + E)$.

Les murs sont appliqués comme suppressions d'arêtes plutôt que comme transformations géométriques [3]. La vérification de connectivité repose sur BFS, avec une complexité $O(V + E)$ sur graphe non pondéré [2]. Les distances de plus court chemin sont aussi réutilisées par les heuristiques IA, notamment pour Minimax et MCTS [4, 5].

En complément, le moteur maintient `player_positions`, deux collections de murs (vertical/horizontal) et un historique de snapshots pour *undo/redo*.

1.3 Déplacements des pions et validation des murs

Les déplacements de base sont obtenus à partir des voisins du sommet courant dans le graphe. Lorsqu'un pion adverse occupe un sommet adjacent, la règle de saut s'applique si la case située derrière reste accessible ; sinon, des déplacements diagonaux sont autorisés selon les arêtes effectivement présentes. Ces cas dépendent de l'état courant des pions et sont traités dynamiquement dans `pawn_rules.py`.

La validation d'un mur combine deux contrôles : validité locale de placement (position, orientation, absence de chevauchement) et maintien d'un chemin vers la zone cible pour chaque joueur. L'implémentation effectue une suppression temporaire des arêtes concernées, exécute un BFS par joueur, puis valide la pose ou restaure l'état précédent. Sur une grille 9×9 , ce coût reste faible.

Le modèle retenu établit un lien direct entre structures de données et règles d'exécution : le graphe représente la connectivité, les règles dynamiques gèrent les cas contextuels et les vérifications globales assurent la validité de la partie.

2 Architecture logicielle

2.1 Architecture générale

Les Figures 1 et 2 présentent les composants principaux du projet en deux vues lisibles.

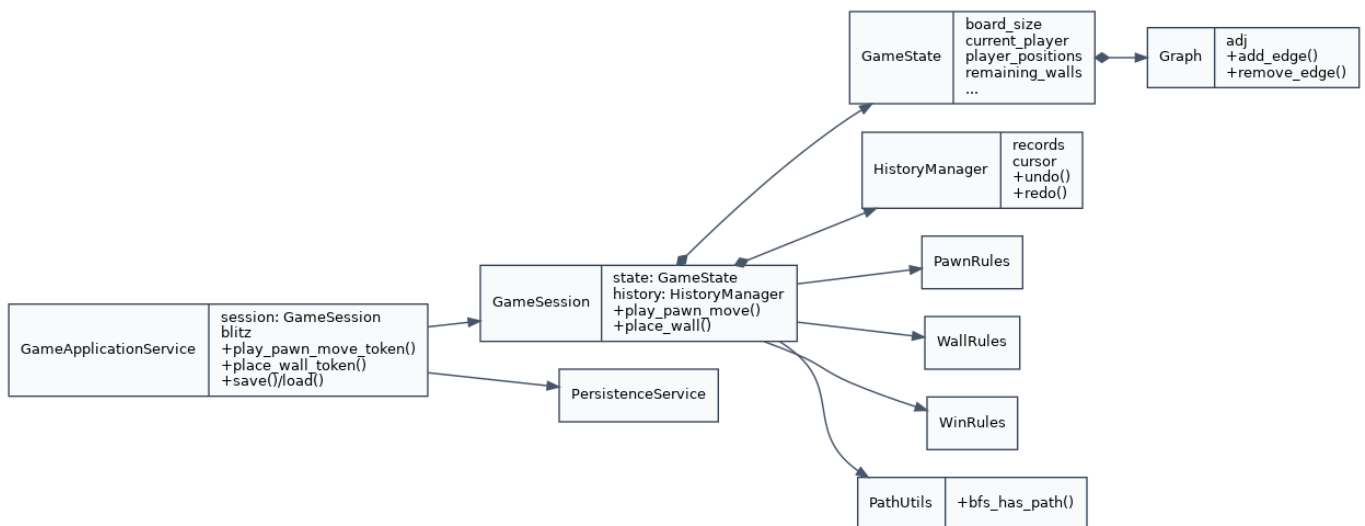


FIGURE 1 – Vue UML du noyau métier et de la couche applicative

L'architecture est organisée en couches : les modules d'interface et de réseau passent par la couche applicative, qui orchestre les opérations métier. `GameState` porte l'état, `GameSession` applique les transitions et `GameApplicationService` constitue le point d'entrée commun pour CLI, GUI, IA, persistance et réseau.

2.2 Modules et dépendances

Le module `core` contient le modèle métier (état, plateau, notation, snapshots). Le module `rules` implémente les validations de déplacements, de murs et de victoire. Le module `application` orchestre les cas d'usage et le cycle de partie. Les modules `interfaces` et `network` gèrent l'entrée/sortie utilisateur et les échanges client/serveur. Le module `utils` regroupe les composants techniques réutilisables (graphe, utilitaires), et `ml` porte les composants d'aide aux moteurs de recherche.

Les dépendances restent dirigées : `interfaces` → `application` → (`core`, `rules`, `utils`), avec `network` connecté à `application`. Cette orientation limite les cycles et isole le coeur métier.

```
quoridor/
|-- core/
|-- rules/
|-- application/
|-- interfaces/
|-- network/
^-- ml/
```

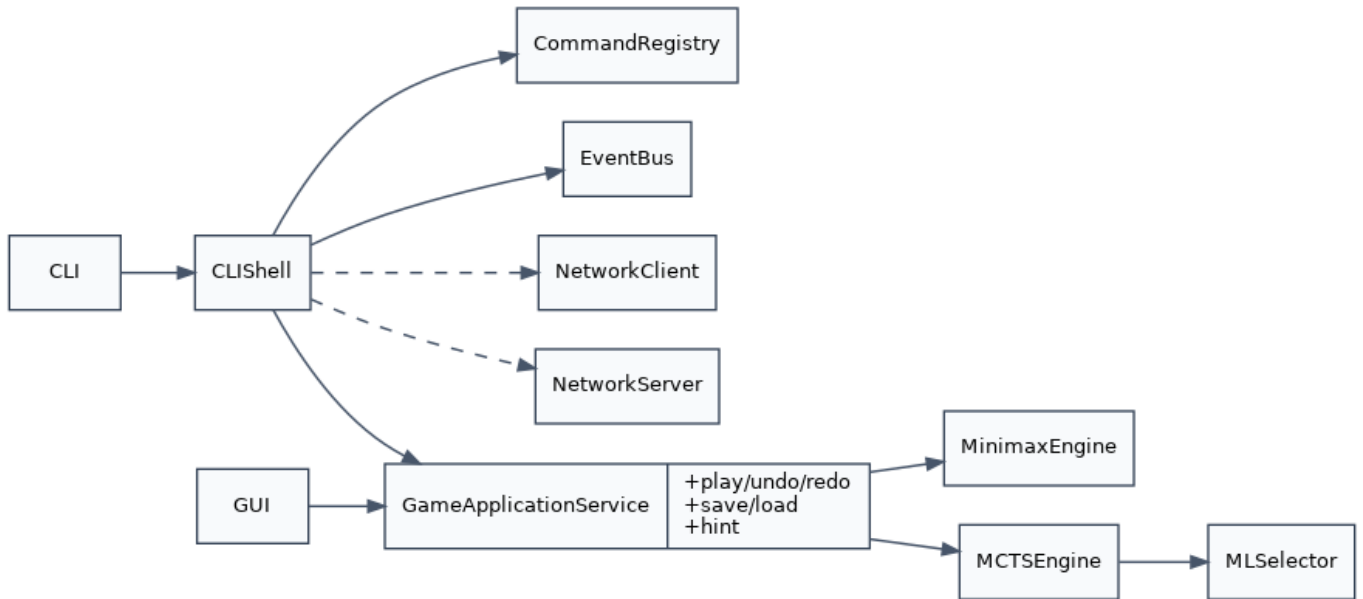


FIGURE 2 – Vue UML des interfaces, du réseau et des composants IA

2.3 Cheminement des actions

Toutes les actions passent par le même pipeline applicatif. Pour un déplacement utilisateur : interface → GameApplicationService → validation rules → mise à jour GameSession/GameState → historisation via HistoryManager → retour interface.

La pose de mur suit la même chaîne, avec simulation sur graphe et test BFS avant validation. Un coup IA suit également ce pipeline : GameApplicationService déclenche Minimax/MCTS, puis applique le coup sélectionné via les mêmes étapes de validation et de mise à jour.

2.4 Intégration des algorithmes

BFS est implémenté dans rules/utils pour les tests de connectivité. Les moteurs Minimax, alpha-bêta et MCTS sont orchestrés par application. Le mécanisme *undo/redo* repose sur HistoryManager et les snapshots d'état. La persistance et le parsing assurent la conversion entre formats externes et représentation interne, sans dupliquer la logique métier.

2.5 Design patterns utilisés

Command Dans le shell CLI, les commandes et CommandRegistry encapsulent les actions utilisateur et délèguent leur exécution à GameApplicationService. Cette structure découple saisie et exécution.

Strategy PlayCommandStrategy, LocalPlayCommandStrategy et NetworkPlayCommandStrategy isolent les variantes d'exécution local/réseau, sans multiplier les conditions dans la couche applicative.

Memento GameSnapshot, MoveRecord et HistoryManager conservent les états avant/après action, ce qui permet des restaurations cohérentes pour *undo/redo*.

Facade GameApplicationService joue le rôle de façade : il expose une API unique pour jouer un coup, poser un mur, gérer l'historique, piloter l'IA et charger/sauvegarder une partie.

Observer EventBus implémente un mécanisme d'observation léger entre producteurs et consommateurs d'événements, utile pour propager les mises à jour sans dépendances directes fortes.

L'ensemble produit une architecture compacte : responsabilités localisées, dépendances orientées vers le métier, et extensions (IA, réseau, interfaces) ajoutées sans modifier le noyau de règles.

Conclusion

Suspendisse sit amet nisi placerat, vestibulum nisl vel, iaculis nunc. Suspendisse quis libero vel ipsum gravida congue nec id arcu. Donec vel mi viverra, commodo urna vitae, tincidunt lectus. Fusce pellentesque, lectus a placerat vestibulum, magna lectus placerat nibh, vehicula pellentesque lacus erat non nulla. Suspendisse maximus, turpis at suscipit egestas, libero massa ultrices libero, at tincidunt velit eros a eros. Maecenas id tempor nunc. Phasellus condimentum porttitor sapien in sagittis. Proin in metus diam. Mauris in turpis sed ex viverra iaculis. Sed lobortis,

tellus vitae congue blandit, leo elit auctor libero, id tempor ante dolor quis quam. Donec eget ligula quam. Maecenas porttitor massa vel felis elementum tristique.

Références

- [1] Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] Paul Glendenning. Quoridor : A graph-theoretic perspective, 2005.
- [4] Isaac Ison. Monte carlo tree search in quoridor, 2018.
- [5] Stephan Mertens. Shortest paths and evaluation functions in quoridor, 2006.
- [6] Stephan Mertens. The game of quoridor : Complexity and strategy, 2012.

A Section en annexe

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.